

Lab 4 — Timing: Hardware Timers and the 1 kHz Problem

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS

Estimated time	2 hrs
Introduces	<code>esp_timer</code> , <code>volatile</code> , measuring instead of trusting
Prerequisites	Lab 2 (tasks and delays).
Beyond the board	A logic analyzer, to measure the period rather than trust it.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§4.1 objective**, **§4.2 parts and wiring**, **§4.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

4.1 Objective & TM4C bridge

Generate a genuinely periodic 1 kHz event that does not depend on the CPU being free, and confirm it on an instrument rather than believing a log line.

TM4C BRIDGE

A bare-metal timer lab has you load a reload value into the GPTM registers, enable the timer interrupt in the NVIC, write the handler, and clear the flag in `ICR`. `esp_timer` replaces that whole sequence with "call this function every N microseconds." The hardware event is identical; the bookkeeping is done for you. Keep the register version in mind, because interviewers ask how a timer works underneath, and you have actually written it.

4.2 Parts & wiring

Item	Status	Connection
ESP32-S3 board	Required	—
Logic analyzer	required this time	GND to board GND (first), CH0 to GPIO4

Two wires from the analyzer and nothing else. This is the lab where the analyzer stops being optional, because the entire point is to *measure* the timing rather than take the firmware's word for it. Sample at **1 MHz**; no decoder is needed since you are measuring a plain square wave.

WARNING
Ground, again

The analyzer is on its own USB cable. Without a wire joining its ground to the board's ground, the two devices have no common reference and every channel reads as a flat or floating line. If your capture looks dead, this is the cause far more often than anything in your code.

4.3 How it works

Three ways to make something happen periodically

Lab 2 established that `vTaskDelay` quantises to the 100 Hz tick, so it cannot produce a 1 kHz event at all. That leaves three options and it is worth seeing why only one is right.

Approach	Accuracy	CPU cost	Verdict
Busy-wait counting loop	Poor — depends on clock speed, compiler optimisation, cache	100% of a core	Never, under an RTOS
RTOS task delay	Limited to the 10 ms tick	Near zero	Fine for slow, soft timing
Hardware timer	Microsecond, independent of CPU load	Near zero	The right tool for periodic sampling

A hardware timer is a counter in silicon, clocked directly from a chip clock. It counts whether or not your code is running, whether or not a higher-priority task is hogging the CPU, and it raises an interrupt when it reaches its target. That independence from software load is the property you are buying, and it is the reason every real sampling system is timer-driven.

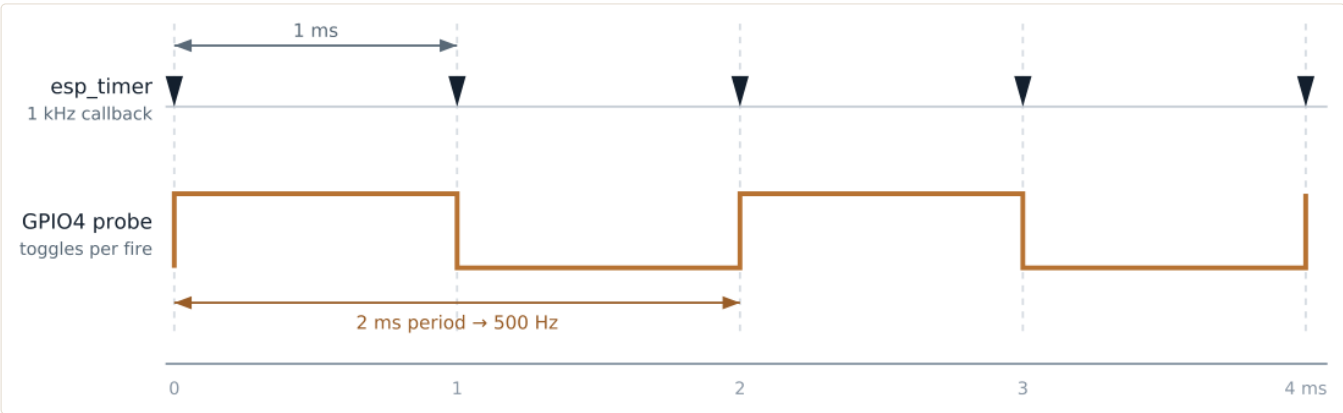


Figure 4.1. Why the probe reads 500 Hz, not 1 kHz. The callback fires every 1 ms, but each firing flips the pin once, so one full cycle of the square wave takes two callbacks.

Timer callback timing versus the resulting square wave.

The `esp_timer` fires a callback every 1 ms (arrows at 0, 1, 2, 3, 4 ms). The GPIO4 probe line toggles once per callback. Because one full cycle of a square wave needs TWO transitions (low-to-high then high-to-low), the resulting waveform has a 2 ms period = 500 Hz.

Measuring 500 Hz on the analyzer therefore CONFIRMS a 1 kHz timer. It is the expected result, not an error.

What `esp_timer` actually is

`esp_timer` is a software layer on top of one hardware timer: it maintains a sorted list of pending timeouts and fires callbacks as each comes due, so you can have many timers from one piece of hardware. By default your callback runs in a dedicated high-priority **task**, not in the raw interrupt itself, which means you may do slightly more inside it than in a bare ISR — though the discipline of keeping it tiny still applies and Lab 7 depends on it.

Its period is specified in **microseconds**. That unit is worth checking in the documentation rather than assuming, because guessing milliseconds here makes your sample rate wrong by a factor of a thousand.

volatile, and why the compiler would otherwise break your code

This is the interview keyword of the lab. A modern compiler optimises aggressively, and one of its favourite tricks is to keep a frequently-read variable in a CPU register instead of re-reading memory. That is correct for ordinary variables, because the compiler can see every write to them.

It is *wrong* for a variable written by a different context. Here `samples` is incremented by the timer callback and read by the main loop. Looking only at the main loop, the compiler sees a variable that is never modified inside it — so it may legitimately read it once, cache it in a register, and use the stale value forever. Your counter would appear frozen.

volatile tells the compiler "this can change behind your back; re-read it from memory every time." Any variable shared with an ISR or callback, and any variable that *is* a hardware register, must be **volatile**. **TM4C bridge:** this is exactly why the register definitions in `tm4c123gh6pm.h` are declared **volatile**, and why any flag your ISR set for `main()` to read needed it too.

WARNING

volatile is not atomicity — a favourite follow-up question

volatile guarantees the compiler will not cache the value. It does **not** make the access indivisible. `samples++` is a read-modify-write: load, add one, store. If another context modifies `samples` between the load and the store, that update is lost. On a single-writer counter like this one it is fine, but the moment two contexts write the same variable you need a real mechanism — a critical section, an atomic type, or a queue. Interviewers ask "does volatile make it thread-safe?" specifically to see whether you know the answer is no.

4.4 Procedure & code

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_timer.h"
#include "driver/gpio.h"
```

```
#include "esp_log.h"

#define PROBE_GPIO 4

static volatile uint32_t samples = 0;

static void sample_cb(void *arg) {           // default dispatch: runs in the esp_timer task
    samples++;
    gpio_set_level(PROBE_GPIO, samples & 1); // square wave at HALF the sample rate
}

void app_main(void) {
    gpio_set_direction(PROBE_GPIO, GPIO_MODE_OUTPUT);
    const esp_timer_create_args_t a = {
        .callback = &sample_cb,
        .name      = "sampler",
    };
    esp_timer_handle_t h;
    esp_timer_create(&a, &h);
    esp_timer_start_periodic(h, 1000);       // 1000 us period = 1 kHz
    while (1) {
        vTaskDelay(pdMS_TO_TICKS(1000));
        ESP_LOGI("lab4", "samples in the last second: %lu", samples);
        samples = 0;
    }
}
```

4.5 Code walkthrough, line by line

Thirty lines, and one keyword in them is a favourite interview trap.

Line 3 · `#include "esp_timer.h"`

The high-resolution timer API. Note it lives under **System** in the documentation rather than Peripherals, because `esp_timer` is a software layer that multiplexes many timers onto one piece of hardware.

Line 9 · `static volatile uint32_t samples = 0;`

Four separate decisions packed into one declaration.

`uint32_t` is a fixed-width 32-bit unsigned integer. Firmware uses fixed-width types as a default because `int`'s size is not guaranteed across compilers and architectures, and when you are counting something at 1 kHz you want to know exactly when it wraps.

`static` keeps it private to this file.

`volatile` is the important one. It tells the compiler "this variable can change behind your back, so never cache it in a register — re-read it from memory every single time." Here `samples` is written by the callback (line 12) and read by the main loop (line 27). Looking only at the loop, the compiler sees a variable that is never modified inside it, so it may legitimately read it once, keep it in a register, and use that stale value forever. Your counter would appear frozen. **TM4C bridge:** this is exactly why the register definitions in `tm4c123gh6pm.h` are declared `volatile`, and why any flag your ISR set for `main()` to read needed it too.

WARNING**The follow-up question you will get**

"Does `volatile` make it thread-safe?" **No**. It guarantees the compiler will not cache the value; it does *not* make the access indivisible. `samples++` on line 12 is a read-modify-write: load, add one, store. If another context modified `samples` between the load and the store, that update would be lost. It is fine here because there is exactly one writer, but the moment two contexts write the same variable you need a critical section, an atomic type, or a queue.

Line 11 · `static void sample_cb(void *arg)`

The callback signature: takes the standard `void *arg` (unused here) and returns nothing. The comment notes that by default `esp_timer` runs callbacks from a dedicated high-priority **task**, not a raw hardware ISR, which is why you may do slightly more here than in a bare interrupt. The discipline of keeping it tiny still applies, and Lab 7 depends on it.

Line 12 · `samples++;`

Counts one tick. See the warning above for why this innocuous line is where the atomicity question lives.

Line 13 · `gpio_set_level(PROBE_GPIO, samples & 1);`

`samples & 1` is a **bitwise AND** with 1, which isolates the lowest bit: it is 1 when `samples` is odd and 0 when it is even. So the pin flips on every callback. This is a common idiom for "is this number odd" and is faster than `% 2` on some compilers, though any modern one optimises both identically.

NOTE**Why a 1 kHz timer gives a 500 Hz wave**

One full cycle of a square wave needs *two* transitions: low-to-high, then high-to-low. Each callback produces one transition, so it takes two callbacks per cycle and the output frequency is exactly half the callback rate. Measuring 500 Hz on the analyzer **confirms** a 1 kHz timer — it is the expected result, not an error.

Line 17 · `gpio_set_direction(PROBE_GPIO, GPIO_MODE_OUTPUT);`

The short form of Lab 1's config struct, and perfectly fine for a single pin with no pull resistor and no interrupt. Use the struct when you need more than direction; use this when you do not.

Lines 18-21 · `const esp_timer_create_args_t a = { .callback = &sample_cb, .name = "sampler" };`

`.callback = &sample_cb` passes the **address of the function**. In C the `&` on a function name is optional — `sample_cb` alone would work identically — but writing it makes the intent explicit. `.name` is used in diagnostics and in `esp_timer_dump()` output, so give it something meaningful.

The struct is declared `const` because the timer API only reads it. Marking things `const` where you can lets the compiler place them in flash rather than RAM, which on a 512 KB part is worth having as a habit.

Lines 22-23 · `esp_timer_handle_t h; esp_timer_create(&a, &h);`

Same pattern as Lab 1's LED strip: you declare a handle, pass its address, and then the create call fills it in. `&a` is the config going in; `&h` is the handle coming out.

Line 24 · `esp_timer_start_periodic(h, 1000);`

Starts it firing every **1000 microseconds**. The unit is the thing to verify in the documentation rather than assume — guessing milliseconds here would make your sample rate wrong by a factor of a thousand, and the code would run perfectly while producing nonsense. `esp_timer_start_once` is the one-shot sibling.

TM4C bridge: this replaces loading a reload value into the GPTM registers, enabling the timer, and enabling its interrupt line in the NVIC. The hardware event is identical; the bookkeeping is done for you.

Lines 26-28 · the reporting loop

The main loop sleeps a full second with `vTaskDelay` — yielding the CPU entirely while the timer keeps running in the background, which is the whole advantage over a busy-wait — then logs the count and resets it.

`%lu` is the conversion for an `unsigned long`, matching `uint32_t`. Using `%d` here would be a type mismatch; on this architecture it happens to work, which is worse than failing, because the habit breaks on a platform where `long` is wider.

Reading roughly 1000 means the timer is keeping real time independently of what the CPU is doing. Reading noticeably less means something is starving the timer task.

4.6 Docs & references

DOCS

Trace every line to its source

`esp_timer_create`, `esp_timer_start_periodic`, `esp_timer_create_args_t` — ESP-IDF API Reference → System → **ESP Timer (High Resolution Timer)**. Go and confirm the period unit is microseconds; this is the canonical fifteen-second lookup that saves a thousand-fold mistake.

`esp_timer_get_time()` — same page. Returns microseconds since boot from a free-running counter. Lab 16 uses it, and it is the right tool whenever you need to measure elapsed time.

`gpio_set_direction`, `gpio_set_level` — Peripherals → GPIO.

The ISR dispatch option — on the same `esp_timer` page, read about `ESP_TIMER_ISR` dispatch. It runs the callback directly in interrupt context for lower latency, with correspondingly stricter rules. Knowing the option exists, and why you would not use it by default, is good interview material.

`volatile` — any C reference, but the embedded-specific framing is what matters. Be ready to explain both what it does and what it does *not* do.

4.7 Verify, troubleshoot, your turn

Verify. The log reports roughly 1000 samples per second. On the analyzer, GPIO4 shows a square wave at about **500 Hz** with a 2 ms period, for the reason in §4.5. Measure the period with cursors

rather than eyeballing it. A reading of 501 Hz rather than exactly 500 is cursor-placement imprecision, not clock error.

Troubleshoot

Symptom	Cause and fix
Count is stuck or nonsensical	Missing <code>volatile</code> , or you forgot to reset the counter.
Count is about 1000 but no wave	Probe on the wrong pin, or missing analyzer ground.
Wave is there but the frequency looks halved	Not a fault — see the two-transitions note.
Count far below 1000	The callback is doing too much work, or something is blocking the timer task.
Build succeeds but old behaviour persists	Stale build cache after copying a project. <code>idf.py fullclean</code> and rebuild.

Your turn

1. Change the period to 500 μ s and confirm on the analyzer that the wave doubles to 1 kHz. You are now calibrating your understanding against an instrument, which is the habit this lab is really teaching.
2. Try to produce the same 1 kHz event with `vTaskDelay` instead and measure what you actually get. You will get about 100 Hz. Record the number in `debug_log.md` — that measurement is the argument for hardware timers, in your own data.
3. Remove `volatile`, build with optimisation, and see whether your compiler caches the read. It may or may not, which is itself the lesson: undefined behaviour is not reliably wrong, it is unreliable.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

Overclaim. `esp_timer` is a *high-resolution software timer*; its callbacks are dispatched from a high-priority ESP Timer task, are serialised with each other, and can be delayed by higher-priority work or by flash operations. §4.7's "keeps real time independently of what the CPU is doing" is too strong. For hard periodicity use **GPTimer**, which is the actual hardware timer peripheral with an ISR-level alarm callback. Also: the main loop's `samples` read-then-reset is not atomic, so counts can be lost between the read and the write — use a critical section or an atomic exchange.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 4.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04 ←	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.